

Intelligent Document Extraction, Validation & API Platform

Neostats AI Engineer Internship - Technical Case Study Solution

- Role: AI Engineer Intern
- Focus: Extraction Completeness, Mathematical Validation, REST APIs, and Deployment
- Tech Stack: Python 3, FastAPI, PyMuPDF, OCR.Space API, SQLite, HTML5/CSS3
- Full compliance with 4 document classes: Invoices, Balance Sheets, P&L, Cash Flow Statements

Problem Statement & Challenge Analysis

Addressing the complexities of modern financial document processing

- **Diversity of Formats:** Organizations receive invoices and financial statements in mixed native and scanned PDF/JPG/PNG formats.
- **Quality & OCR Degradation:** Varying image resolutions, skewed layouts, and table structures create severe OCR noise.
- **Strict Financial Relationships:** Extraction without validation is dangerous in financial operations; numbers must reconcile mathematically.
- **Deployment & API Mandate:** Evaluators require a zero-friction live deployment, OpenAPI docs, persistent dashboard, and structured JSON.

End-to-End System Architecture

A decoupled, 6-tier pipeline engineered for reliability and observability

- Tier 1 - Client & Ingestion: Web Dashboard and REST API accepting multipart/form-data uploads.
- Tier 2 - Input Validation Gate: Enforces PDF/JPG/PNG MIME types, file integrity, non-empty payload, and ≤ 3 pages limit.
- Tier 3 - Dual OCR & Text Engine: PyMuPDF raster rendering (150 DPI) + OCR.Space REST API + Gemini Vision multimodal fallback.
- Tier 4 - Entity Extraction & Grounding: Schema-guided parser extracting line items, multi-year comparative periods, and source text events.
- Tier 5 - Financial Validation Engine: Automated mathematical reconciliation across all 4 document domains with variance tolerance.
- Tier 6 - Persistent Store & Delivery: SQLite ACID database storing full payloads, surfaced via FastAPI and Dashboard.

Document Validation & Input Control Gate

Rigorous first-line defense before heavy model / OCR invocation

- MIME & Extension Enforcement: Strict acceptance of PDF, JPG, PNG; instant rejection of invalid extensions (.exe, .txt, .docx).
- Zero-Byte & Corruption Detection: Validates file buffer length and uses PIL Image.verify() and PyMuPDF doc.rect checks.
- Page Limit Constraint: Enforces ≤ 3 pages maximum; automatically aborts with PAGE_LIMIT_EXCEEDED if violated.
- Controlled Error Envelope: Standardized JSON error response with HTTP 400 Bad Request and descriptive diagnostic codes.

OCR & Field Extraction with Evidence Grounding

Complete line-item extraction without hallucinations

- Native vs Scanned Detection: Inspects embedded text vectors; seamlessly switches to 150 DPI page rasterization for scanned files.
- Evidence & Traceability: Every critical field includes `source_text` and `page_number` so auditors can trace numbers back to the document.
- Confidence Scoring: Explainable confidence metric (0.95 - 0.99) based on OCR character fidelity and regex pattern validation.
- Missing Field Discipline: System returns null for absent values rather than inferring or hallucinating imaginary numbers.

Financial Calculation & Validation Engine

Domain-specific accounting reconciliation with tolerance control

- Invoices: $\text{Quantity} \times \text{Unit Price} \cdot \text{Line Total}; \text{Line Sums} \cdot \text{Subtotal}; \text{Subtotal} + \text{Tax} - \text{Discount} \cdot \text{Total}; \text{Cash Paid} - \text{Total} \cdot \text{Change}.$
- Balance Sheets: $\text{Total Capital \& Liabilities} \cdot \text{Total Assets}; \text{Sum of liability components} \cdot \text{Reported Total}; \text{Sum of assets} \cdot \text{Reported Total}.$
- Profit & Loss: $\text{Interest} + \text{Other Income} \cdot \text{Total Income}; \text{Expended} + \text{Operating} + \text{Provisions} \cdot \text{Total Expenditure}; \text{Income} - \text{Exp} \cdot \text{Net Profit}.$
- Cash Flow: $\text{Operating} + \text{Investing} + \text{Financing} + \text{FX} + \text{Amalgamation} \cdot \text{Net Increase in Cash}; \text{Opening} + \text{Net Increase} \cdot \text{Closing Cash}.$
- Tolerance & Null Safety: Evaluates with 0.05 numerical tolerance; returns NOT_APPLICABLE if required operands are absent.

REST API Design & OpenAPI Documentation

Production-grade FastAPI endpoints with interactive Swagger UI

- `POST /api/v1/documents/process:` Accepts multipart file and `document_type`; returns complete structured JSON synchronously.
- `GET /api/v1/documents/{document_name}:` Fetches latest stored extraction, file validation, and validation audit trail.
- `GET /api/v1/documents:` Returns list of processed document records for the dashboard table.
- `GET /api/v1/health:` System health monitoring endpoint returning status healthy, timestamp, and version.
- Interactive Swagger Docs: Fully browsable and executable at `/docs`, OpenAPI 3.0 schema at `/openapi.json`.

Frontend Dashboard & Persistent Storage

Modern, intuitive financial analyst user experience

- **Interactive Dashboard:** Document upload form with drag-and-drop file input, category dropdown, and real-time processing spinner.
- **Processed Documents Table:** Live dashboard connected to SQLite database displaying status badges, confidence, and page count.
- **Detailed Inspection View:** Formula breakdown cards, operands, variances, extracted line-item tables, and missing field alerts.
- **Raw JSON Viewer:** Full structured JSON inspector with one-click Copy to Clipboard and direct API links.

Deployment Strategy & Production Roadmap

Cloud readiness for Render / Railway / Koyeb and enterprise scaling

- Zero-Friction Cloud Deployment: Configured with Procfile, runtime.txt, and dynamic PORT binding for Render / Railway / Koyeb.
- Secret Hygiene: Zero hardcoded secrets; configuration loaded strictly through environment variables (.env.example).
- Production Roadmap - Asynchronous Queue: Migration to Celery / Redis worker architecture for high-volume document ingestion.
- Production Roadmap - Blob Storage: Offloading uploads and raster renders to AWS S3 or Google Cloud Storage (GCS).
- Production Roadmap - Model Finetuning: Fine-tuning layout-aware multimodal models (e.g., LayoutLMv3) for specialized tax forms.